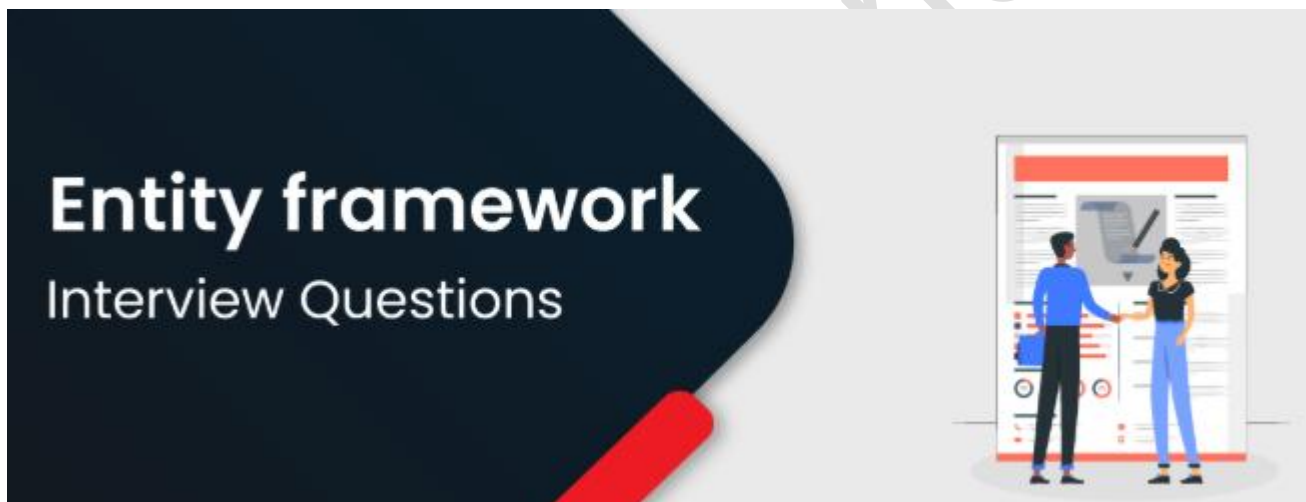




Dot Net Full Stack Developer

Interview Questions and Answers

Dot Net Full Stack Developer Interview Q&A

Q1: What is Entity Framework?

A: Entity Framework (EF) is an Object-Relational Mapper (ORM) developed by Microsoft that allows developers to interact with a relational database using .NET objects. It eliminates the need to write most of the data-access code manually.

EF lets you query and save data to a database using C# or VB.NET instead of SQL queries.

Benefits:

- Reduces boilerplate SQL code
- Supports LINQ for querying
- Manages relationships, migrations, and change tracking

Q2: What are the different approaches to use EF?

A: EF supports the following approaches:

1. **Code First:** You define classes and EF generates the database schema.
2. **Database First:** You start with an existing database, and EF generates models (classes).
3. **Model First** (EF 6 only): You design the model visually (EDMX), and EF creates database and classes.

Common Use Case:

- **Code First** is popular in modern apps.
- **Database First** is used when working with legacy databases.

Q3: How do you perform CRUD operations in EF?

A: EF allows you to perform Create, Read, Update, and Delete using DbContext.

```
using (var context = new AppDbContext())
{
    // CREATE
    var user = new User { Name = "John" };
    context.Users.Add(user);
    context.SaveChanges();

    // READ
    var users = context.Users.ToList();

    // UPDATE
    var userToUpdate = context.Users.First();
    userToUpdate.Name = "Updated";
    context.SaveChanges();

    // DELETE
```

```

var userToDelete = context.Users.First();
context.Users.Remove(userToDelete);
context.SaveChanges();
}

```

Q4: What is DbContext? What is its role?

A: DbContext is the **primary class** in EF for interacting with the database. It acts as a bridge between your domain classes and the database.

Responsibilities of DbContext:

- Managing entity objects during runtime
- Querying the database
- Tracking changes
- Saving data (via SaveChanges())

```

public class AppDbContext : DbContext
{
    public DbSet<User> Users { get; set; }
}

```

Q5: What is DbSet in Entity Framework?

A: DbSet<T> represents a **table** in the database. Each DbSet allows you to query and work with data from that table.

```

public class AppDbContext : DbContext
{
    public DbSet<User> Users { get; set; } // Maps to Users table
}

```

Example:

```

var allUsers = context.Users.ToList(); // SELECT * FROM Users

```

Q6: What is the difference between Add, AddRange, and Attach?

A:

Method	Description
Add()	Marks a single entity for insertion into the DB.
AddRange()	Adds multiple entities at once.
Attach()	Attaches an existing entity to context in Unchanged state (useful for updates when tracking is off).

Example:

```

context.Users.Add(new User { Name = "New" });
context.Users.AddRange(listOfUsers);
context.Users.Attach(existingUser); // No insert/update, just track

```

Q7: How does EF handle relationships (one-to-many, many-to-many)?

A: EF maps **relationships** using **navigation properties** and **foreign keys**.

One-to-Many:

```
public class Blog
{
    public int BlogId { get; set; }
    public List<Post> Posts { get; set; } // One Blog -> Many Posts
}
```

```
public class Post
{
    public int PostId { get; set; }
    public int BlogId { get; set; } // FK
    public Blog Blog { get; set; }
}
```

Many-to-Many (EF Core 5+):

```
public class Student
{
    public int Id { get; set; }
    public ICollection<Course> Courses { get; set; }
}

public class Course
{
    public int Id { get; set; }
    public ICollection<Student> Students { get; set; }
}
```

EF auto-creates the join table.

Q8: What is lazy loading, eager loading, and explicit loading?

A:

Loading Type	Description
Lazy Loading	Loads related data when accessed .
Eager Loading	Loads related data immediately using .Include() .
Explicit Loading	Manually loads data using Entry().Reference().Load()

Example:

```
// Eager loading
var blog = context.Blogs.Include(b => b.Posts).ToList();
```



```
// Lazy loading (requires setup)
var post = context.Posts.First();
var blog = post.Blog; // triggers DB call
```

```
// Explicit loading
var blog = context.Blogs.First();
context.Entry(blog).Collection(b => b.Posts).Load();
```

Q9: What is the purpose of OnModelCreating method?

A: OnModelCreating is a method in DbContext used to configure model behavior using Fluent API.

Used for:

- Table and column mappings
- Keys, constraints
- Relationships
- Value conversions
- Filters

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .HasKey(u => u.UserId);

    modelBuilder.Entity<User>()
        .Property(u => u.Name)
        .HasMaxLength(100)
        .IsRequired();
}
```

Q10: How do you configure a primary key and foreign key using data annotations?

A: Primary Key:

```
public class User
{
    [Key]
    public int Id { get; set; }
}
```

Foreign Key:

```
public class Post
{
    public int Id { get; set; }
    [ForeignKey("Blog")]
    public int BlogId { get; set; }
}
```

```
public Blog Blog { get; set; }  
}
```

Q11: What is the difference between EF Core and EF 6?

A: EF Core is a lightweight, extensible, and cross-platform rewrite of Entity Framework 6. It supports modern development patterns and runs on .NET Core and .NET 5+. EF 6, on the other hand, is the full-featured framework designed for .NET Framework (also supported on .NET 6+ in limited ways).

Key differences:

- **Cross-platform:** EF Core works on Windows, Linux, macOS; EF 6 is Windows-only.
- **Performance:** EF Core offers better performance and faster LINQ translations.
- **Features:** EF Core supports features like shadow properties, batch updates, alternate keys, global filters, and value conversions which EF6 doesn't.
- **Migrations:** EF Core has improved and simplified migration tooling.
- **Many-to-Many:** EF Core 5+ supports many-to-many without needing a join entity, whereas EF6 requires it.

Q12: How does EF Core handle migrations? What is the purpose of Add-Migration and Update-Database?

A: EF Core uses **migrations** to manage schema changes in a Code-First approach. A migration is a way to apply changes to your model (classes) and sync them with the database schema.

- **Add-Migration <name>:** Creates a new migration class that contains the schema changes based on changes in your DbContext or entity classes.
- **Update-Database:** Applies the migration to the actual database, creating or altering tables/columns.

Example:

Add-Migration InitialCreate

Update-Database

You can also generate SQL scripts using:

Script-Migration

EF Core stores migration history in the __EFMigrationsHistory table.

Q13: How can you seed initial data using EF Core?

A: EF Core allows you to seed data in the OnModelCreating() method using the Fluent API. This is different from manually inserting records during runtime.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)  
{  
    modelBuilder.Entity<User>().HasData(  
        new User { Id = 1, Name = "Admin" },  
        new User { Id = 2, Name = "Guest" }  
    );  
}
```

}

When you run Update-Database, EF will insert this data if it doesn't exist.

Note: All seeded data must have a primary key defined.

You can also reseed programmatically via `context.Users.AddRange(...)`, but that's considered runtime seeding.

Q14: How do you write raw SQL queries or stored procedure calls in EF?

A: EF Core allows you to run raw SQL queries using `FromSqlRaw()` and `ExecuteSqlRaw()` methods. These are useful when you need optimized SQL, use stored procedures, or when LINQ is too limiting.

Query Example:

```
var users = context.Users
```

```
    .FromSqlRaw("SELECT * FROM Users WHERE Name = {0}", "John")  
    .ToList();
```

Stored Procedure Example:

```
var users = context.Users
```

```
    .FromSqlRaw("EXEC GetUsersByRole @Role={0}", "Admin")  
    .ToList();
```

Non-query command:

```
context.Database.ExecuteSqlRaw("DELETE FROM Users WHERE IsActive = 0");
```

Note: `FromSqlRaw` must be used on `DbSet<T>` and only supports SELECT-type queries.

Q15: How does EF track changes in entities? What is the ChangeTracker?

A: EF Core tracks the state of each entity using the **Change Tracker**, which determines what changes have been made to the objects since they were loaded or added.

States include:

- Added
- Unchanged
- Modified
- Deleted
- Detached

EF compares original and current values to generate SQL during `SaveChanges()`.

Example:

```
var user = context.Users.First();  
user.Name = "Updated";
```

```
var changes = context.ChangeTracker.Entries()  
    .Where(e => e.State != EntityState.Unchanged);
```

You can also disable tracking for better performance using `AsNoTracking()` if you don't need change detection.

Q16: What is the AsNoTracking() method and when would you use it?

A: AsNoTracking() tells EF not to track the returned entities. This improves performance, especially in read-only scenarios like reporting or dashboards.

Example:

```
var users = context.Users.AsNoTracking().ToList();
```

EF skips the overhead of change detection, which is useful when you just want to read data without updating it.

Use cases:

- Display-only views
- Export data
- API responses with large data sets

If you need to update the entity later, you must manually attach it again.

Q17: What are shadow properties in EF Core?

A: Shadow properties are properties defined in the EF model but **not present in your C# entity class**. They're useful for tracking metadata like CreatedDate, LastModifiedBy, etc.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .Property<DateTime>("CreatedAt");
}
```

You can set/get values like:

```
context.Entry(user).Property("CreatedAt").CurrentValue = DateTime.Now;
```

These properties exist only in the database and EF model, not in your class definition.

Q18: How do you handle concurrency in EF Core?

A: EF Core handles **optimistic concurrency** using a concurrency token like a RowVersion or Timestamp. EF checks this value when updating and throws a DbUpdateConcurrencyException if it has changed since it was fetched.

Example:

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; }
}
```

During SaveChanges(), EF includes the original RowVersion in the WHERE clause of the update. If no row is affected, EF knows the data was changed by someone else.

You can catch the exception and handle it accordingly:

```
catch (DbUpdateConcurrencyException ex)
{
    // Log, Retry, or Alert user
}
```

Q19: Explain the use of Fluent API vs Data Annotations.

A: Both Fluent API and Data Annotations are used to configure EF Core mappings, constraints, and relationships.

Data Annotations:

- Easy to use
- Decorates the model directly

[Key]

[MaxLength(100)]

public string Name { get; set; }

Fluent API:

- More powerful and flexible
- Keeps configuration out of domain classes

```
modelBuilder.Entity<User>()
```

```
.Property(u => u.Name)
```

```
.HasMaxLength(100)
```

```
.IsRequired();
```

Best Practice: Use Data Annotations for simple rules; use Fluent API for complex configurations (composite keys, relationships, value conversions, filters, etc.).

Q20: What is Global Query Filter in EF Core?

A: Global Query Filters are applied automatically to all queries for a given entity. They are useful for **multi-tenancy, soft deletes, or authorization**.

Example – Soft Delete:

```
public class User
```

```
{
```

```
    public int Id { get; set; }
```

```
    public bool IsDeleted { get; set; }
```

```
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

```
{
```

```
    modelBuilder.Entity<User>().HasQueryFilter(u => !u.IsDeleted);
```

```
}
```

Now every time you run:

```
context.Users.ToList();
```

EF will internally add WHERE IsDeleted = false.

Q21: How do you implement unit of work and repository patterns with EF?

A: EF Core already provides a **unit of work pattern** via DbContext and **repository-like behavior** via DbSet<T>, but you can still abstract them for better separation of concerns.

Repository Interface:

```
public interface IRepository<T> where T : class
{
    T GetById(int id);
    IEnumerable<T> GetAll();
    void Add(T entity);
    void Remove(T entity);
}
```

Repository Implementation:

```
public class Repository<T> : IRepository<T> where T : class
{
    private readonly AppDbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repository(AppDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public T GetById(int id) => _dbSet.Find(id);
    public IEnumerable<T> GetAll() => _dbSet.ToList();
    public void Add(T entity) => _dbSet.Add(entity);
    public void Remove(T entity) => _dbSet.Remove(entity);
}
```

Unit of Work:

```
public interface IUnitOfWork : IDisposable
{
    IRepository<User> Users { get; }
    int Complete();
}
```

```
public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;
```



```
public IRepository<User> Users { get; private set; }
```

```
public UnitOfWork(AppDbContext context)
{
    _context = context;
    Users = new Repository<User>(_context);
}
```

```
public int Complete() => _context.SaveChanges();
public void Dispose() => _context.Dispose();
}
```

Q22: What are some performance optimization techniques in EF?

A: Here are some key ways to improve EF Core performance:

1. **Use AsNoTracking()** for read-only queries to avoid tracking overhead.

```
var users = context.Users.AsNoTracking().ToList();
```

2. **Select only required columns** with .Select() projection.

```
var names = context.Users.Select(u => u.Name).ToList();
```

3. **Use eager loading (Include) carefully**, avoid unnecessary joins.

```
var blogs = context.Blogs.Include(b => b.Posts).ToList();
```

4. **Avoid N+1 problems** (see Q26).

5. **Batch operations** instead of looping individual Add() or Remove().

6. **Use compiled queries** for repetitive complex LINQ:

```
private static readonly Func<AppDbContext, string, User> _getUserByName =
    EF.CompileQuery((AppDbContext context, string name) =>
        context.Users.FirstOrDefault(u => u.Name == name));
```

7. **Paginate large queries:**

```
var users = context.Users.Skip(100).Take(50).ToList();
```

8. **Use caching** (EF does not cache queries natively, consider second-level caching).

Q23: When should you use Dapper over EF Core and why?

A: Dapper is a micro ORM that is faster and better suited for raw SQL access and performance-critical scenarios.

Use **Dapper** when:

- You need maximum performance (e.g., large SELECTs or batch inserts).
- You want full control over SQL and stored procedures.
- You are not using complex relationships (no change tracking or navigation).

Use **EF Core** when:

- You need change tracking, migrations, and LINQ-based querying.
- You want easy-to-use model binding and rich ORM features.

Example Dapper usage:

```
using (var conn = new SqlConnection(_connStr))
{
    var users = conn.Query<User>("SELECT * FROM Users WHERE IsActive = 1").ToList();
}
```

EF is great for **developer productivity**, while Dapper is great for **performance and SQL control**.

Q24: What are the drawbacks of EF? How do you mitigate them?

A:

Drawback	Mitigation
Slower performance than raw SQL	Use AsNoTracking(), Dapper for hotspots
Difficult debugging of generated SQL	Use ToQueryString() or SQL Profiler
N+1 problem	Use .Include() or projection queries
Large models may slow startup	Split into bounded contexts
Abstraction can hide performance issues	Profile with logging or EF.Diagnostics

EF is very powerful, but being **aware of its internal behavior** (e.g., lazy loading, joins, state tracking) helps you avoid common pitfalls.

Q25: How does EF handle transactions? Can you explicitly manage them?

A: Yes, EF Core supports both **automatic** and **explicit transactions**.

By default, SaveChanges() is wrapped in a transaction.

You can manage transactions manually using BeginTransaction().

```
using var transaction = context.Database.BeginTransaction();
```

```
try
{
    context.Users.Add(new User { Name = "A" });
    context.SaveChanges();

    context.Posts.Add(new Post { Title = "Hello" });
    context.SaveChanges();

    transaction.Commit();
}
catch
{
    transaction.Rollback();
}
```

You can also use TransactionScope for distributed transactions.

EF ensures **ACID** compliance by allowing rollback on failure.

Q26: Explain the N+1 query problem and how to avoid it in EF.

A: The **N+1 problem** occurs when EF executes one query to load a parent record and N separate queries to load related child records, one per parent.

Example:

```
var blogs = context.Blogs.ToList(); // 1 query
foreach (var blog in blogs)
{
    var posts = blog.Posts; // Triggers 1 query per blog => N queries
}
```

Solution 1: Eager Loading

```
var blogs = context.Blogs.Include(b => b.Posts).ToList(); // 1 JOIN query
```

Solution 2: Select projection

```
var blogs = context.Blogs
    .Select(b => new { b.Title, Posts = b.Posts.Select(p => p.Title) })
    .ToList();
```

Avoiding N+1 improves performance, especially when dealing with many records.

Q27: How does EF Core support value converters?

A: Value converters transform data between the **CLR type** and the **database type**. They are useful for enums, encrypted strings, custom types, etc.

Example: Enum to string converter

```
public enum Status { Active, Inactive }
public class User
{
    public int Id { get; set; }
    public Status Status { get; set; }
}
```

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .Property(u => u.Status)
        .HasConversion<string>();
}
```

You can also use custom converters:

```
.HasConversion(
    v => Encrypt(v),
    v => Decrypt(v)
)
```

EF Core applies the conversion during read/write operations automatically.

Q28: Can you map database views in EF Core?

A: Yes, EF Core allows mapping **database views** to entities using `HasNoKey()` since views usually don't have primary keys.

Example:

```
public class UserSummary
{
    public string Name { get; set; }
    public int PostCount { get; set; }
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<UserSummary>()
        .HasNoKey()
        .ToView("View_UserSummary");
}
```

Then you can query it like a table:

```
var summaries = context.Set<UserSummary>().ToList();
```

Note: Views are **read-only** unless they are updateable and EF can determine the key.

Q29: How can you implement soft delete with EF Core?

A: Soft delete means marking records as deleted without physically removing them.

Add a flag to your model:

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public bool IsDeleted { get; set; }
}
```

Apply Global Query Filter:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>().HasQueryFilter(u => !u.IsDeleted);
}
```

Override `SaveChanges` to handle delete:

```
public override int SaveChanges()
{
    foreach (var entry in ChangeTracker.Entries<User>())
    {
        if (entry.State == EntityState.Deleted)
```

```

    {
        entry.State = EntityState.Modified;
        entry.Entity.IsDeleted = true;
    }
}
return base.SaveChanges();
}

```

This way, delete operations mark the entity as deleted but don't remove it from the database.

Q30: How does EF Core handle connection pooling?

A: EF Core relies on **ADO.NET connection pooling**, which is enabled by default when using SQL Server or other supported providers.

Connection pooling reuses database connections instead of opening/closing on each call, improving performance and reducing overhead.

To configure it, use `AddDbContextPool<TContext>()` for **DbContext pooling** in DI: `services.AddDbContextPool<AppDbContext>(options =>`

`options.UseSqlServer(configuration.GetConnectionString("Default")));`

This allows EF Core to reuse instances of `AppDbContext`, which further improves memory and CPU usage in high-load apps.

Connection pooling settings like Max Pool Size can be controlled via the connection string:

`Server=...;Database=...;Trusted_Connection=True;Max Pool Size=100;`

Both connection and DbContext pooling reduce initialization overhead and are recommended in web applications.

Q31: How do you manage large domain models with EF Core?

A: Large domain models can lead to performance, maintainability, and startup time issues.

To manage them in EF Core, you can use:

1. **Modular DbContexts** (Bounded Contexts): Break your large model into smaller DbContexts aligned to business modules.

```
public class BillingContext : DbContext { public DbSet<Invoice> Invoices { get; set; } }
```

```
public class HRContext : DbContext { public DbSet<Employee> Employees { get; set; } }
```

2. **Entity Configuration Classes:** Move Fluent API into separate configuration files.

```
public class UserConfiguration : IEntityTypeConfiguration<User>
```

```

{
    public void Configure(EntityTypeBuilder<User> builder)
    {
        builder.Property(u => u.Name).HasMaxLength(100);
    }
}

```

3. **Use Assembly Scanning:**

modelBuilder.ApplyConfigurationsFromAssembly(typeof(UserConfiguration).Assembly);

4. **Avoid loading unnecessary navigation properties** with Include or projection queries.
5. **Use partial classes** and **feature folders** to organize large models cleanly.

Q32: How do you design multi-tenant applications using EF Core?

A: Multi-tenancy in EF Core can be achieved using:

1. **TenantId field in all entities:**

```
public class Order { public int TenantId { get; set; } }
```

2. **Global Query Filters** to isolate data per tenant:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>().HasQueryFilter(o => o.TenantId == _currentTenantId);
}
```

3. **Custom DbContext per tenant (Schema-per-tenant):**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasDefaultSchema(_tenantSchema);
}
```

4. **Custom IModelCacheKeyFactory** when model changes per tenant (see Q37).
5. Store tenant context using scoped service or middleware for multi-tenant resolution.

Approach depends on whether you're using:

- **Single DB per tenant**
- **Shared DB with tenant ID (preferred for SaaS)**
- **Separate schema per tenant**

Q33: Can you integrate CQRS with EF Core? How?

A: Yes, you can integrate CQRS (Command Query Responsibility Segregation) with EF Core by separating **read and write concerns**.

1. **Use separate models and contexts:**

- Command model uses full EF with change tracking.
- Query model uses read-only DTOs via AsNoTracking() or raw SQL.

```
public class CommandDbContext : DbContext { }
```

```
public class QueryDbContext : DbContext { }
```

2. **MediatR pattern** for command/query handlers:

```
public record CreateUserCommand(string Name) : IRequest<int>;
```

```
public class CreateUserHandler : IRequestHandler<CreateUserCommand, int>
```

```
{
    private readonly CommandDbContext _context;
    public Task<int> Handle(CreateUserCommand cmd, CancellationToken ct) { ... }
}
```

3. **Projection queries** for reads:


```
var users = _queryContext.Users.AsNoTracking()
    .Select(u => new UserDto { Name = u.Name }).ToList();
```

4. Can use **Dapper for queries** and **EF Core for commands** in high-performance systems.

CQRS helps improve scalability and maintainability in microservices and large-scale systems.

Q34: How do you profile and debug EF-generated SQL?

A: You can profile EF Core-generated SQL using multiple techniques:

1. Use **ToQueryString()** in EF Core 5+:

```
var query = context.Users.Where(u => u.Age > 25);
Console.WriteLine(query.ToQueryString());
```

2. Enable logging in DbContext:

```
optionsBuilder.LogTo(Console.WriteLine)
    .EnableSensitiveDataLogging();
```

3. Use EF Profiler tools like:

- MiniProfiler
- EF Core Power Tools
- JetBrains dotTrace

4. **SQL Server Profiler** or **Azure Query Insights** to monitor database-side performance.
5. Add logging providers (e.g., Serilog) for production-ready logging of EF queries.

Q35: Have you used EF Core in microservices? What are the challenges?

A: Yes, EF Core can be used in microservices with careful design. **Challenges include:**

1. **Schema ownership:** Each microservice should own its own schema and DbContext.
2. **Database per service:** Encouraged to maintain isolation but complicates reporting.
3. **Cross-service queries:** You can't perform joins across services. Use APIs or async messaging (e.g., Kafka, RabbitMQ).
4. **Data duplication:** Common for read models (eventual consistency).
5. **Migrations:** Each service needs its own migration pipeline in CI/CD.

Solutions:

- Apply **bounded contexts** and **CQRS**.
- Use **outbox pattern** for event publishing.
- Use **DbContext pooling** for performance.
- Prefer Dapper for lightweight services.

Q36: How do you handle schema evolution/versioning in large teams?

A: In large teams, schema changes must be managed carefully.

1. Use **EF Core migrations** with version control:

Add-Migration AddIsActiveToUsers

2. **Avoid automatic migration in production** (EnsureCreated only for dev/testing).
3. **Separate migration projects** per bounded context.
4. **Use a branching strategy** (like GitFlow) to manage concurrent migration conflicts.

5. **Generate migration SQL scripts:**

dotnet ef migrations script -o migration.sql

6. Apply DB versioning tools like:

- Flyway
- Liquibase
- DbUp

7. Automate with CI/CD pipelines for rollback-safe deployments.

Q37: Can you implement a custom IModelCacheKeyFactory for multi-tenancy?

A: Yes. When your model changes **per tenant** (e.g., schema-per-tenant), you must implement IModelCacheKeyFactory so EF Core rebuilds the model accordingly.

Example:

```
public class TenantModelCacheKeyFactory : IModelCacheKeyFactory
{
    public object Create(DbContext context)
    {
        if (context is AppDbContext appContext)
            return (context.GetType(), appContext.TenantId);
        return context.GetType();
    }
}
```

Register it in DI:

```
services.AddSingleton<IModelCacheKeyFactory, TenantModelCacheKeyFactory>();
```

Without this, EF reuses the model from the first tenant, causing schema or filter mismatches.

Q38: How do you enforce data security at the EF level (row-level security)?

A: To implement **row-level security**, use **Global Query Filters** and/or database-level policies.

1. **EF Global Filters:**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>().HasQueryFilter(o => o.UserId == _currentUserId);
}
```

2. **Database-level RLS (SQL Server):**

```
CREATE SECURITY POLICY OrderFilter
```

```
ADD FILTER PREDICATE dbo.fn_order_filter(UserId) ON dbo.Orders
```

```
WITH (STATE = ON);
```

3. Combine both for defense-in-depth:

- EF filters for dev-side enforcement.
- DB-side filters for runtime security.

4. Use middlewares or scoped services to inject current user/tenant ID.

Q39: How do you handle large data imports/exports with EF?

A: Large data operations in EF can be slow due to change tracking. Use the following optimizations:

1. **Disable tracking:**

```
context.ChangeTracker.AutoDetectChangesEnabled = false;
```

2. **Use AddRange/BulkInsert:**

```
context.Users.AddRange(largeList);
```

```
context.SaveChanges();
```

3. **Batch SaveChanges** in chunks:

```
foreach (var chunk in largeList.Chunk(1000))
```

```
{
```

```
    context.Users.AddRange(chunk);
```

```
    context.SaveChanges();
```

```
}
```

4. **Use third-party bulk extensions:**

- EFCore.BulkExtensions
- Z.EntityFramework.Extensions

5. **Export** via:

- LINQ-to-CSV or JSON serialization
- Streaming large results using yield return

For ETL processes, Dapper or direct ADO.NET might be better suited.

Q40: How do you decide between using raw SQL, Dapper, and EF Core?

A: The choice depends on use case, performance, complexity, and control required.

Use EF Core when:

- You need change tracking, migrations, and relationship mapping.
- Developer productivity is a priority.
- Application is domain-driven.

Use Dapper when:

- Performance is critical (bulk read, joins).
- You write custom SQL or work with stored procedures.
- You only need lightweight mapping without tracking.

Use Raw SQL in EF Core when:

- You need precise control for a few complex queries.

```
context.Users.FromSqlRaw("SELECT * FROM Users WHERE IsActive = 1").ToList();
```

Summary:

- EF Core → for general-purpose CRUD and domain modeling
- Dapper → for performance-sensitive operations
- Raw SQL → for occasional special-case queries

Hybrid approach (EF + Dapper) is often used in real-world enterprise apps.

Q41: What are best practices to organize EF Core code in a clean architecture solution?

A: In a Clean Architecture solution, EF Core code should be organized to **preserve separation of concerns**, keeping infrastructure code (EF Core) out of core business logic.

Best practices:

1. **Domain Layer** (Entities only):
 - Contains POCO classes without EF attributes or dependencies.
2. **Application Layer**:
 - Contains interfaces like IRepository, IUnitOfWork, and use cases.
 - No EF Core reference here.
3. **Infrastructure Layer**:
 - Implements DbContext, repositories, and EF configurations.
 - Uses Fluent API to configure models.
4. **Web/API Layer**:
 - Injects application services and works with DTOs/ViewModels.

Folder structure:

- Domain
 - Entities
- Application
 - Interfaces
 - DTOs
- Infrastructure
 - Data
 - DbContext
 - Migrations
 - Repositories
- WebAPI
 - Controllers

Use DbContext only in the infrastructure layer to ensure clean dependency boundaries.

Q42: What are the risks of using EF in a high-performance application?

A: EF Core is powerful but may introduce performance risks if not used carefully:

Common risks:

1. **Over-fetching data:** Caused by lazy loading or unfiltered queries.
2. **N+1 query problem:** Occurs when related data isn't eagerly loaded.
3. **Tracking overhead:** EF tracks all entities by default.
4. **Memory pressure:** Especially with large datasets and tracking enabled.
5. **Slow migrations:** Especially with complex schema updates in production.

Mitigations:

- Use .AsNoTracking() for read-only queries.
- Use projection queries (Select) to reduce payload.

- Monitor generated SQL with ToQueryString() or logging.
- Use Dapper or stored procedures for read-heavy operations.
- Avoid unnecessary joins and includes.

Q43: How do you manage migrations across environments (Dev/Test/Prod)?

A: Migrations should be applied consistently across environments using version control and deployment scripts.

Best practices:

1. Version migrations in source control – never auto-generate on prod.
2. Generate migration scripts:

`dotnet ef migrations script -o migrate.sql`

3. Apply migrations in CI/CD pipeline using:
 - EF CLI
 - SQL script execution
 - DbUp or other migration tools
4. Use environment-based configs for connection strings.
5. Tag migration points using naming conventions like AddUsers_Prod_2025.

Q44: How do you prevent accidental data loss during automatic migrations?

A: Automatic migrations can drop or alter tables/data unintentionally.

Prevention methods:

1. Avoid EnsureDeleted() or EnsureCreated() in production.
2. Never enable Migrate() on app startup in Prod without review.
3. Use Add-Migration and review migration files carefully.
4. Generate scripts instead of applying directly:

`dotnet ef migrations script -o safe-migrate.sql`

5. Backup databases before applying changes.
6. Use feature toggles or shadow tables for high-risk schema changes.

Q45: What is the role of DbContextFactory in EF Core?

A: IDbContextFactory<T> is used to **create instances of DbContext manually**, outside of the dependency injection scope.

Use cases:

- Background services (e.g., hosted services, workers).
- Blazor WASM apps (no scoped DI).
- Factory-based multi-tenancy.

Example:

public class Worker : BackgroundService

{

private readonly IDbContextFactory<AppDbContext> _factory;

public Worker(IDbContextFactory<AppDbContext> factory) => _factory = factory;


```
protected override async Task ExecuteAsync(CancellationToken stoppingToken)
{
    using var context = _factory.CreateDbContext();
    // use context
}
}
```

Register in DI:

```
services.AddDbContextFactory<AppDbContext>(options => ...);
```

Q46: How do you handle connection resilience and retry policies with EF Core?

A: EF Core supports **built-in retry policies** using `EnableRetryOnFailure()` for transient failures (e.g., timeouts, deadlocks).

Example:

```
options.UseSqlServer(connStr, sqlOptions =>
{
    sqlOptions.EnableRetryOnFailure(
        maxRetryCount: 5,
        maxRetryDelay: TimeSpan.FromSeconds(10),
        errorNumbersToAdd: null);
});
```

Also:

- Wrap calls in Polly retry policies for additional control.
- Monitor for `SqlException` numbers related to transient errors.

Q47: How does EF Core interact with distributed cache?

A: EF Core does not provide distributed caching out of the box, but you can implement caching at the application layer using services like:

- Redis
- Memcached
- `MemoryCache` (for small apps)

Example using Redis with EF query results:

```
var cachedUsers = await _cache.GetAsync("users");
if (cachedUsers == null)
{
    var users = await _context.Users.ToListAsync();
    await _cache.SetAsync("users", users);
}
```

Use:

- **`IMemoryCache`** for local in-memory cache
- **`IDistributedCache`** for scalable caching
- **Cache aside pattern** for cache population and invalidation

Avoid caching tracked EF entities directly—always cache DTOs.

Q48: How do you tune performance when dealing with millions of records?

A: When working with large datasets, EF Core needs tuning to avoid timeouts and memory exhaustion.

Techniques:

1. **Pagination:**

```
context.Users.Skip(page * 1000).Take(1000).ToList();
```

2. **Batch processing:**

```
foreach (var batch in largeList.Chunk(1000))
{
    context.Users.AddRange(batch);
    context.SaveChanges();
}
```

3. **Use .AsNoTracking()** for read-heavy operations.

4. **Indexing** in database to speed up query execution.

5. **Split queries for large joins:**

```
options.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery);
```

6. **Dapper** for bulk read scenarios.

7. **EFCore.BulkExtensions** for BulkInsert, BulkUpdate, etc.

8. Use projections to fetch only required fields.

Q49: Explain how to test EF Core code using in-memory database vs mocking.

A: EF Core supports testing with **InMemoryDatabase** and mocking DbSet with interfaces.

In-memory testing:

```
options.UseInMemoryDatabase("TestDb");
```

- Pros: Fast, easy to set up.
- Cons: Doesn't behave exactly like relational DB (e.g., no constraints, joins).

Mocking:

- Use IDbContext and IRepository interfaces with Moq or FakeItEasy.
- Mocking DbSet<T> can be difficult due to LINQ behavior.

Example:

```
var context = new TestDbContext(options);
context.Users.Add(new User { Name = "Test" });
context.SaveChanges();
```

Q50: What are the limitations of EF Core and how do you work around them?

A: EF Core is powerful but has limitations. Here's how to handle some common ones:

Limitation	Workaround
Limited control over SQL	Use FromSqlRaw or Dapper
No lazy loading (unless enabled)	Use .Include() or explicit loading

Limitation	Workaround
InMemory provider not enforcing constraints	Use SQLite in-memory provider
Complex queries may perform poorly	Use compiled queries or raw SQL
Limited batch update/delete	Use EFCore.BulkExtensions or raw SQL
Lack of true second-level cache	Use third-party cache libraries
Not good fit for reporting/data warehouse	Use Dapper, SQL views, or SPs

Q51: You have a large table with 100M rows. How would you retrieve paginated data efficiently?

A: Efficient pagination on large datasets requires optimizing queries to avoid performance issues due to large OFFSET values.

Best practices:

1. **Keyset Pagination (Seek Method):**

Instead of using Skip, filter using indexed columns (e.g., ID or timestamp).

```
var pageSize = 100;
```

```
var lastId = 100000; // from previous page
```

```
var page = context.Users
```

```
    .Where(u => u.Id > lastId)
```

```
    .OrderBy(u => u.Id)
```

```
    .Take(pageSize)
```

```
    .AsNoTracking()
```

```
    .ToList();
```

This is **much faster** than using .Skip(N) for deep pages.

2. **Ensure proper indexes** on pagination columns (Id, CreatedDate, etc.).

3. **Avoid using OFFSET alone** for deep pages:

```
// Only good for small datasets
```

```
context.Users.OrderBy(u => u.Id).Skip(500000).Take(50);
```

4. **Use stored procedures or SQL views** for complex paging logic.

Q52: A developer pushes a migration that drops a critical table. How would you recover?

A: To recover from a destructive migration:

Immediate steps:

1. **Restore from backup** (prod environments should always have daily/hourly backups).

2. **Revert migration:**

```
dotnet ef migrations remove
```

3. **Generate reverse migration manually** to recreate the table:

```
migrationBuilder.CreateTable(
```

```
    name: "CriticalTable",
```

```
    ...
```

);

4. **Review migration history:**

Check __EFMigrationsHistory table and Git history to identify faulty migration.

5. **Prevent future issues:**

- Enforce PR reviews and code reviews.
- Run migrations in staging first before prod.
- Protect critical tables with database-level permissions.

Q53: You notice a slow-performing query from EF. How do you identify and optimize it?

A: Steps to troubleshoot and optimize:

1. Log and Inspect Generated SQL:

```
Console.WriteLine(query.ToQueryString());
```

2. **Use SQL Profiler or Query Store** to analyze slow queries.

3. **Enable EF logging:**

```
optionsBuilder.LogTo(Console.WriteLine).EnableSensitiveDataLogging();
```

4. **Check for:**

- N+1 issues → fix with .Include() or projections.
- Missing indexes → add indexes to filtered/sorted columns.
- Fetching unnecessary columns → use .Select() to project.
- Large navigation loads → split queries or reduce joins.

5. **Apply AsNoTracking()** for read-only operations.

6. **Use compiled queries** for repeat queries:

```
EF.CompileQuery((ctx, name) => ctx.Users.FirstOrDefault(u => u.Name == name));
```

7. **Profile DB stats:** Use execution plans in SSMS or EXPLAIN.

Q54: How do you enforce business rules like soft delete, auditing, or tenant isolation using EF Core?

A: EF Core supports enforcing such rules using **global filters** and **intercepting SaveChanges()**.
Soft Delete:

```
public class Entity { public bool IsDeleted { get; set; } }
modelBuilder.Entity<Entity>().HasQueryFilter(e => !e.IsDeleted);
public override int SaveChanges()
{
    foreach (var entry in ChangeTracker.Entries<Entity>())
    {
        if (entry.State == EntityState.Deleted)
        {
            entry.State = EntityState.Modified;
            entry.Entity.IsDeleted = true;
        }
    }
}
```

```

    return base.SaveChanges();
}
Auditing:
public override int SaveChanges()
{
    var now = DateTime.UtcNow;
    foreach (var entry in ChangeTracker.Entries())
    {
        if (entry.Entity is IAuditable auditable)
        {
            if (entry.State == EntityState.Added)
                auditable.CreatedAt = now;
            if (entry.State == EntityState.Modified)
                auditable.UpdatedAt = now;
        }
    }
    return base.SaveChanges();
}

```

Tenant Isolation:

```
modelBuilder.Entity<Order>().HasQueryFilter(o => o.TenantId == _currentTenantId);
```

Store TenantId and UserId in scoped services or middleware.

Q55: How would you integrate EF Core in a Clean Architecture with MediatR and Unit of Work?

A: Structure:

- **Domain Layer:** Plain POCOs (no EF references).
- **Application Layer:** MediatR requests/handlers, use IRepository, IUnitOfWork.
- **Infrastructure Layer:** Implements DbContext, Repositories, UoW.

Example MediatR command:

```

public record CreateUserCommand(string Name) : IRequest<int>;
public class CreateUserHandler : IRequestHandler<CreateUserCommand, int>
{
    private readonly IUnitOfWork _uow;

    public CreateUserHandler(IUnitOfWork uow) => _uow = uow;

    public async Task<int> Handle(CreateUserCommand request, CancellationToken ct)
    {
        var user = new User { Name = request.Name };
        _uow.Users.Add(user);
    }
}

```



```

        await _uow.CompleteAsync();
        return user.Id;
    }
}

```

UnitOfWork:

```

public interface IUnitOfWork
{
    IUserRepository Users { get; }
    Task<int> CompleteAsync();
}

```

EF Core's DbContext is instantiated in the **Infrastructure layer**, injected using AddDbContext() or IDbContextFactory<T>.

Q56: Explain how you would implement optimistic concurrency using rowversion.

A: Optimistic concurrency ensures that updates happen only if data hasn't changed since it was last read.

Step-by-step:

1. Add a RowVersion property:

```

public class Product
{
    public int Id { get; set; }

    [Timestamp]
    public byte[] RowVersion { get; set; }
}

```

EF will include RowVersion in the WHERE clause during update/delete. If no rows are affected, it throws DbUpdateConcurrencyException.

Catch and resolve:

```

try
{
    context.SaveChanges();
}
catch (DbUpdateConcurrencyException ex)
{
    foreach (var entry in ex.Entries)
    {
        var dbValues = entry.GetDatabaseValues();
        var currentValues = entry.CurrentValues;
    }
}

```

```
// Decide whether to keep current, overwrite, or merge  
}  
}
```

4. Use RowVersion as a hidden field in UI to prevent update conflicts on user edits. This mechanism avoids locking and supports scalability in multi-user systems.

